Sprint 4: Creación de Base de Datos

Santiago Álvarez Salvado

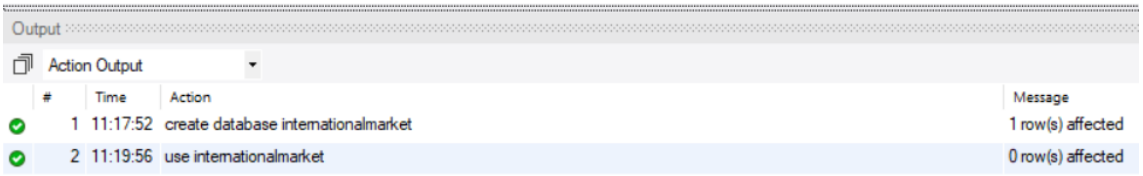
Contenido

Nivel 1

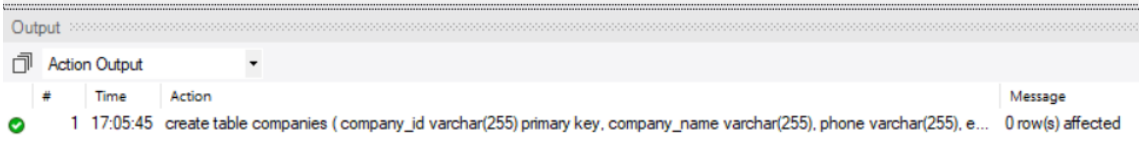
Descarga los archivos CSV, estúdialos y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:

Al estudiar los csv se puede observar que la mayoría están separados por comas, menos el csv transactions que está separado por punto y coma, se procede a crear la base de datos (internationalmarket) las tablas en base a los datos de cada csv, poniendo como tipo de dato varchar(255) para evitar errores, más adelante se ajustan los tipos y longitudes viendo realmente la longitud con max(length()), aunque como se especificó en el sprint 3, en un entorno real no se ponen los datos ajustados al máximo actual, si no que se ponen la longitud más grande por si entra algún dato que exceda la longitud actual y no cause errores.

```
7 • create database internationalmarket;
8 • use internationalmarket;
```




```
10 • create table companies (
11       company_id varchar(255) primary key,
12       company_name varchar(255),
13       phone varchar(255),
14       email varchar(255),
15       country varchar(255),
16       website varchar(255)
17       );
```



Para los usuarios se creara una tabla común llamada users, ya que si se ponen las dos tablas (users europeos y americanos) se rompería el esquema en estrella, también se añade un campo (user_continent) para especificar si pertenece al continente europeo o al americano, a la hora de introducir los datos también se especifica si es usuario europeo o americano con set user_continent = ue si es eruopeo o usa si es americano.

```sql
create table users (
    id varchar(255) primary key,
    name varchar(255),
    surname varchar(255),
    phone varchar(255),
    email varchar(255),
    birth_date varchar(255),
    country varchar(255),
    city varchar(255),
    postal_code varchar(255),
    address varchar(255),
    user_continent enum('EU', 'USA') not null
    );
```

Output

Action Output

#	Time	Action	Message
1	19:50:39	create table users (id varchar(255) primary key, name varchar(255), surname varchar(255), phone varchar(255), email va...	0 row(s) affected

Las demás tablas se crean de la misma manera.

```sql
create table products (
    id varchar(255) primary key,
    product_name varchar(255),
    Price varchar(255),
    color varchar(255),
    weight varchar(255),
    warehouse_id varchar(255)
    );
```

Output

Action Output

#	Time	Action	Message
1	19:50:39	create table users (id varchar(255) primary key, name varchar(255), surname varchar(255), phone varchar(255), email va...	0 row(s) affected
2	19:52:14	create table products (id varchar(255) primary key, product_name varchar(255), Price varchar(255), color varchar(255), ...	0 row(s) affected

Se crea la tabla credit_card del mismo modo que las anteriores (el campo user_id no se declara como clave foránea ya que esto causaría una relación circular al estar user_id relacionado con transaction, con lo cual el campo user_id de la tabla credit_card se podría eliminar)

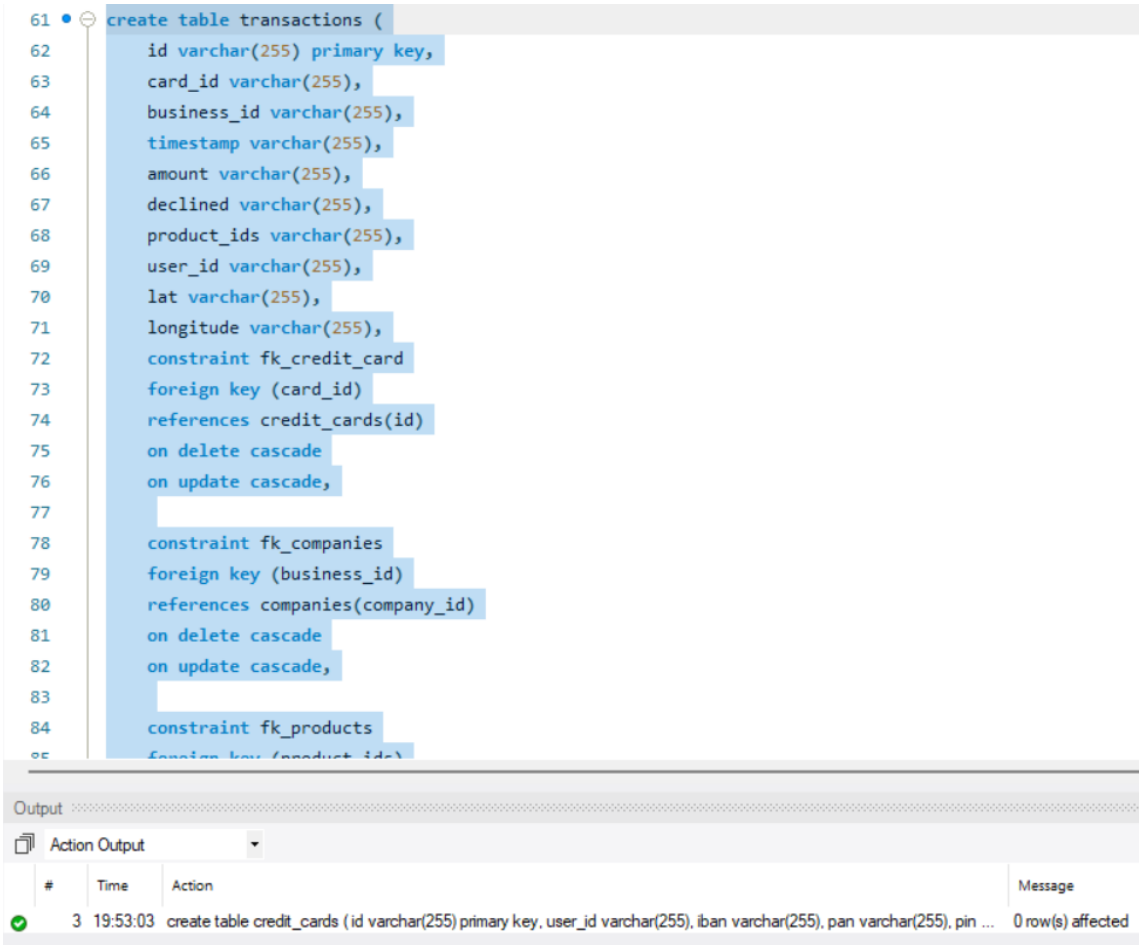
```sql
create table credit_cards (
    id varchar(255) primary key,
    user_id varchar(255),
    iban varchar(255),
    pan varchar(255),
    pin varchar(255),
    cvv varchar(255),
    track1 varchar(255),
    track2 varchar(255),
    expiring_date varchar(255)
    );
```

Output

Action Output

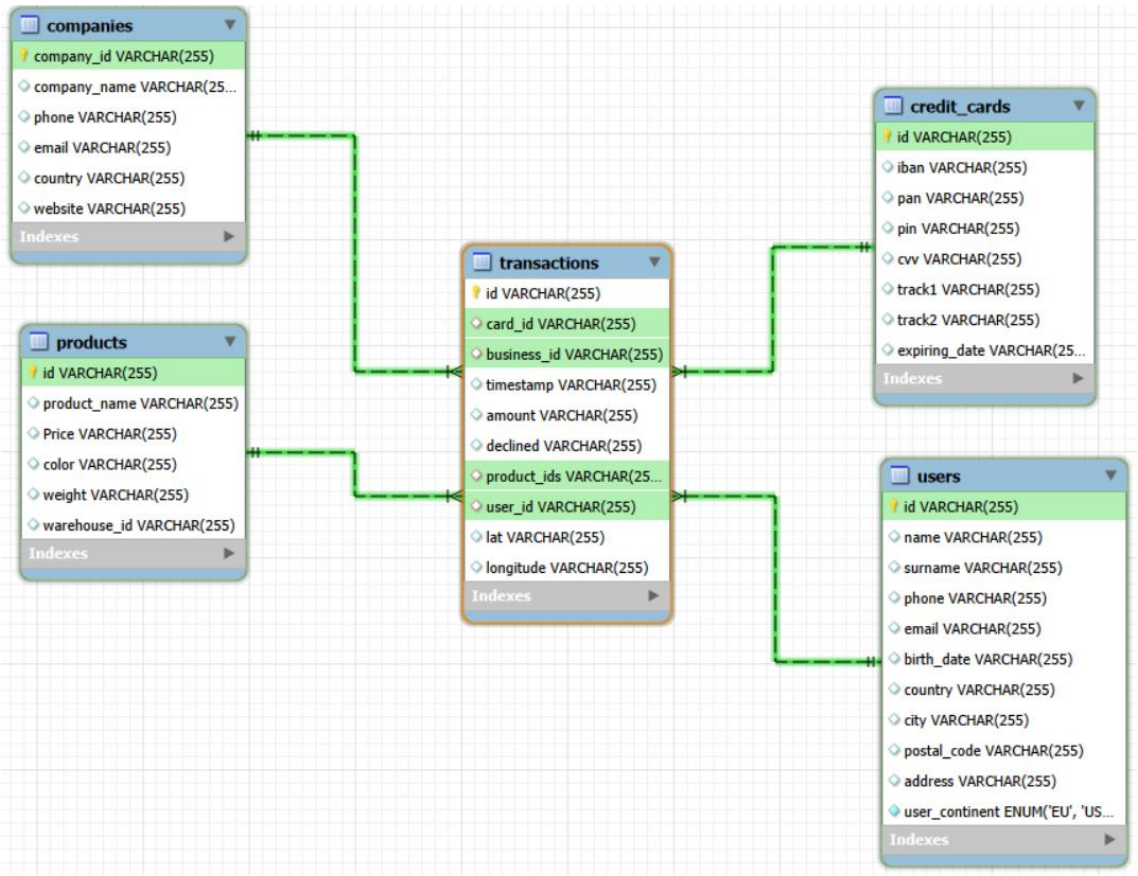
#	Time	Action	Message
3	19:53:03	create table credit_cards (id varchar(255) primary key, user_id varchar(255), iban varchar(255), pan varchar(255), pin ...	0 row(s) affected

En la tabla transactions se definen las claves foráneas de credit_cards con card_id, companies con business_id, products con product_ids y users con user_id, también se definen con on delete cascade y on update cascade para que al eliminar o actualizar en las tablas padre se elimine o actualice en las tablas hijas, es decir, si se elimina un registro en la tabla users por ejemplo, también se eliminara en la tabla transactions, de esta manera se evitaran los registros huérfanos.



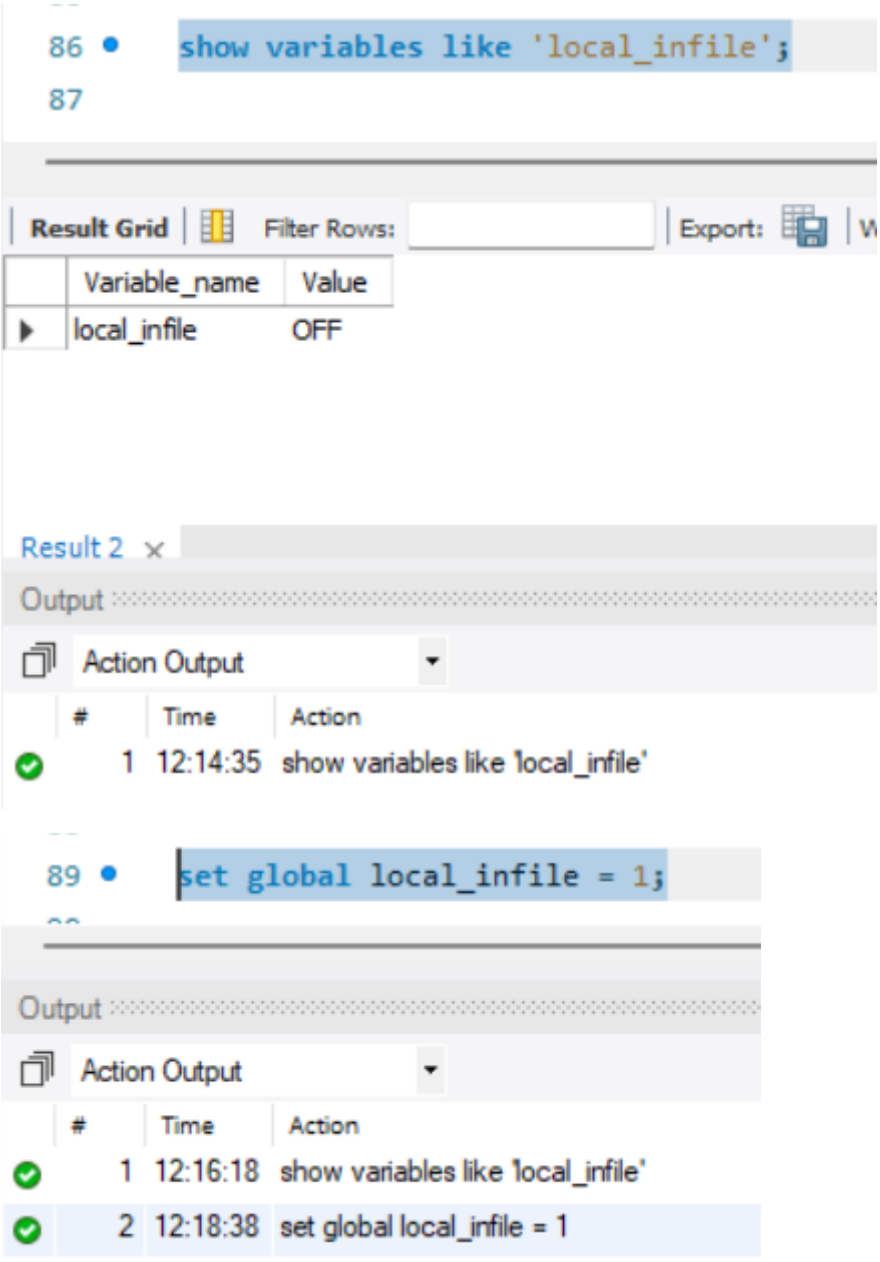
```
61  create table transactions (
62      id varchar(255) primary key,
63      card_id varchar(255),
64      business_id varchar(255),
65      timestamp varchar(255),
66      amount varchar(255),
67      declined varchar(255),
68      product_ids varchar(255),
69      user_id varchar(255),
70      lat varchar(255),
71      longitude varchar(255),
72      constraint fk_credit_card
73      foreign key (card_id)
74      references credit_cards(id)
75      on delete cascade
76      on update cascade,
77
78      constraint fk_companies
79      foreign key (business_id)
80      references companies(company_id)
81      on delete cascade
82      on update cascade,
83
84      constraint fk_products
```

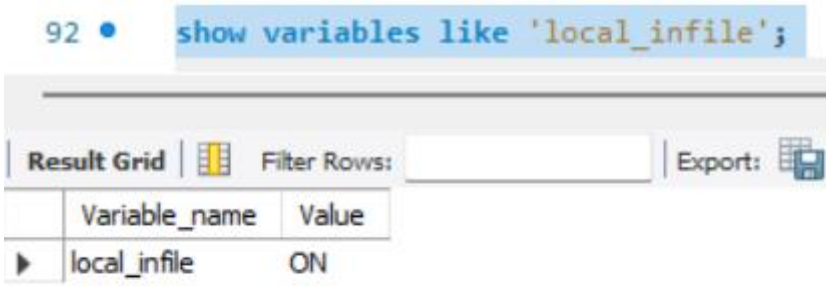
```
Output
Action Output
#   Time      Action                                                                                  Message
3   19:53:03  create table credit_cards ( id varchar(255) primary key, user_id varchar(255), iban varchar(255), pan varchar(255), pin ...  0 row(s) affected
```

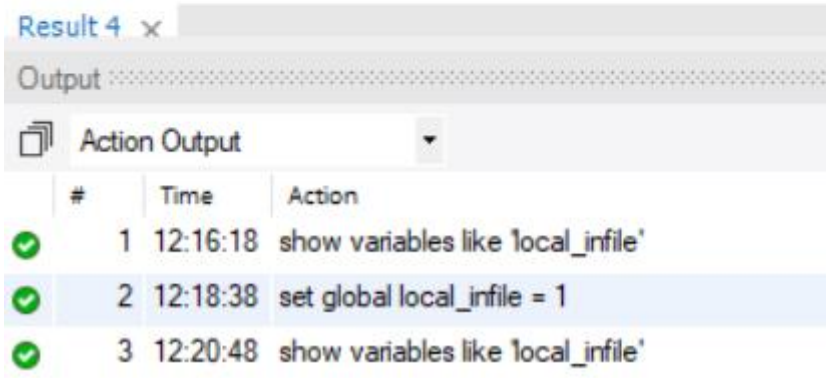


El diagrama quedaría así, se puede observar que es un modelo en estrella, todas las tablas se relacionan con transactions y todas las relaciones son de 1-N desde las tablas (companies, products, credit_cards y users) hacia transactions; la clave foránea del campo user_id en la tabla credit_card no se ha creado ya que como se ha explicado más arriba, eso daría lugar a una relación circular e innecesaria ya que users y credit_card se pueden relacionar a través de transactions.

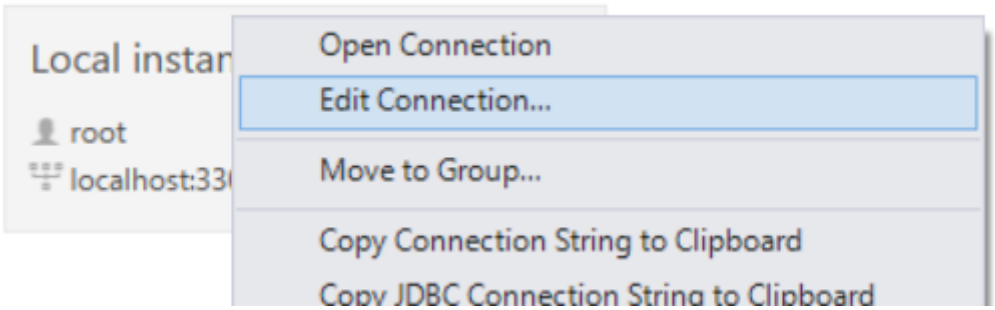
Para poder cargar los datos no se puede utilizar load data infile porque mysql está protegido, por lo que se realizara con load data local infile, pero el servidor debe permitirlo, para ello se comprobara si está activado o no con show variables like ‘local_infile’; en value pondrá on/off, si está en off habrá que poner set global local_infile = 1;

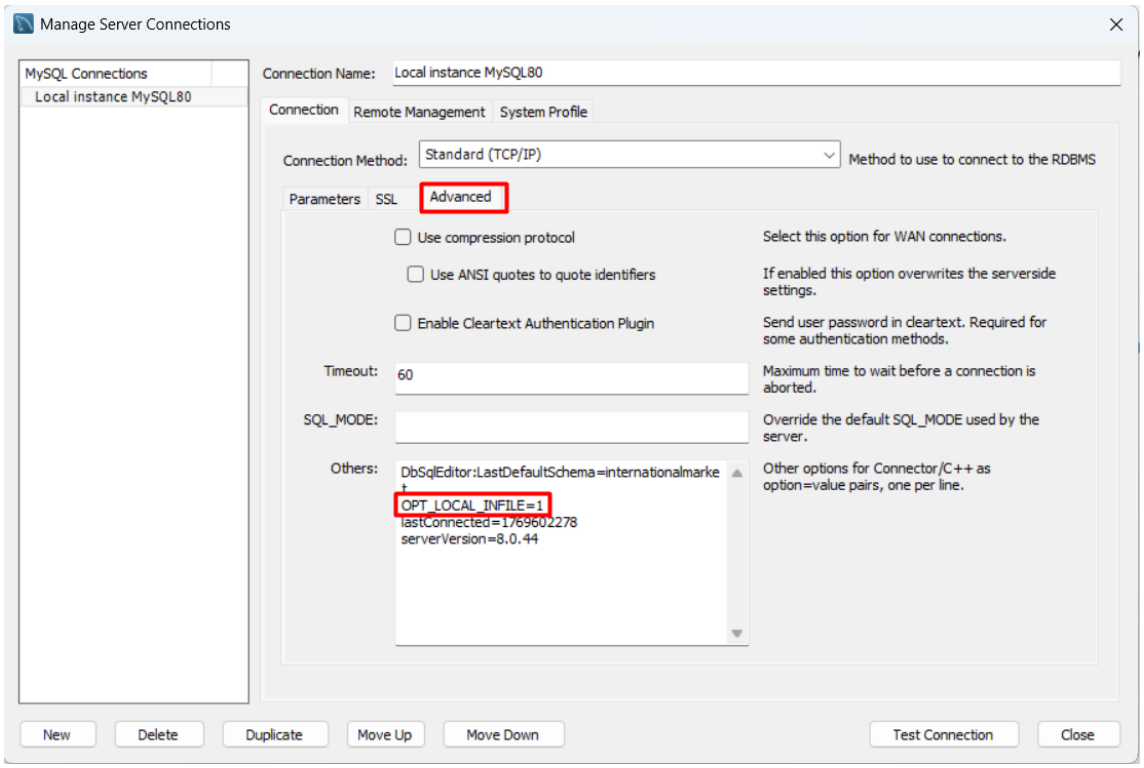




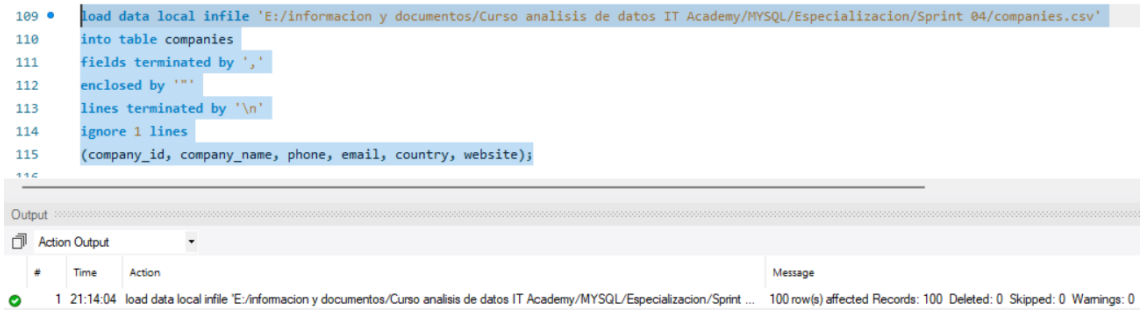


Una vez activada la opción de cargar localmente se cierra workbench, se vuelve a abrir, en la conexión será necesario darle a Edit Connection…, dentro habrá que darle a Advanced y poner en el cuadro de texto inferior OPT_LOCAL_INFILE=1 y se realiza la conexión de nuevo en el servidor.

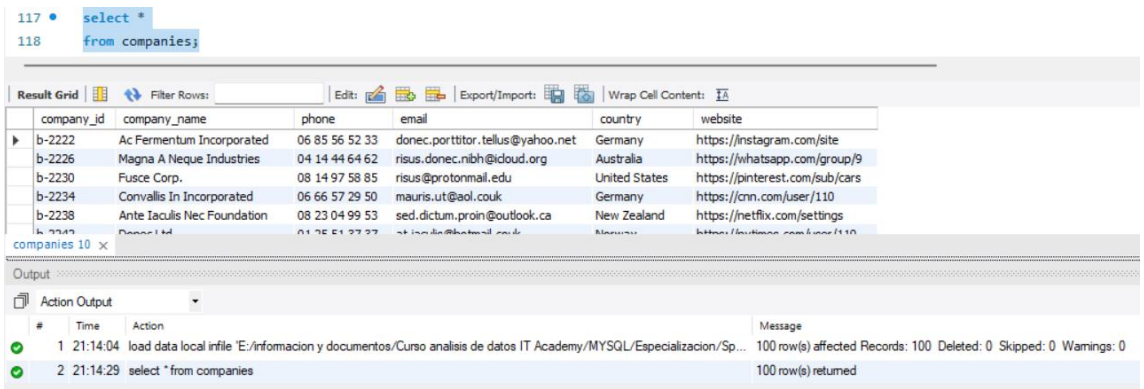




Se repite la ejecución de la query para cargar los datos y ahora funciona sin problema.



Se verifica que los datos del csv companies.csv se han introducido correctamente en la tabla companies.



Al cargar los datos de los usuarios, primero se utilizará el csv european_users.csv y en el set se especificara que estos usuarios pertenecen a EU en el campo user_continent.

```
load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Sprint 04/european_users.csv'
into table users
fields terminated by ','
enclosed by '"'
lines terminated by '\n'
ignore 1 lines
(id, name, surname, phone, email, birth_date, country, city, postal_code, address)
set user_continent = 'EU';
```

Output

Action Output

#	Time	Action	Message
1	21:14:04	load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Sp...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0
2	21:14:29	select * from companies	100 row(s) returned
3	21:15:59	load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Sp...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0

Se verifica que los datos del csv european_users.csv se han insertado correctamente en la tabla users.

```
select *
from users;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	name	surname	phone	email	birth_date	country	city	postal_code	address	user_continent
1000	Amkjrv	Qbulrxbp	+48-258-9936	amkjrv.qbulrxbp@example.com	May 17, 1970	Germany	Stuttgart	70173	215 Qbulrxbp St	EU
1001	Nfvrlb	Oydaiwbg	+94-121-2522	nfvrlb.oydaiwbg@example.com	Mar 4, 1994	Germany	Cologne	50667	121 Oydaiwbg St	EU
1002	Ijbfmd	Jbddzhvp	+70-120-3668	ijbfmd.jbddzhvp@example.com	Sep 27, 2001	Germany	Munich	80331	412 Jbddzhvp St	EU
1003	Uyciig	Sfbdymzj	+58-123-6968	uyciig.sfbdymzj@example.com	Jan 20, 1981	Germany	Stuttgart	70173	735 Sfbdymzj St	EU
1004	Yjqurq	Ojizvgqi	+77-944-2340	yjqurq.ojizvgqi@example.com	Jul 27, 1954	Germany	Munich	80331	685 Ojizvgqi St	EU
1005	Mnlqtu	Glofegwk	+54-801-2627	mnlqtu.glofegwk@example.com	Nov 15, 1962	Portugal	Funchal	9000-001	8 Glofegwk St	EU
1007	Ehcrlp	Xahkzrlm	+71-862-6101	ehcrlp.xahkzrlm@example.com	Nov 8, 1971	Spain	Sevilla	41001	205 Xahkzrlm St	EU
1008	Securp	Faofvqfy	+76-822-2041	securp.faofvqfy@example.com	Nov 13, 1957	Sweden	Stockholm	111 20	535 Faofvqfy St	EU
1009	Rsznah	Vfnffant	+67-294-3155	rsznah.vfnffant@example.com	Nov 10, 1952	Spain	Valencia	46001	507 Vfnffant St	EU

users 11

Output

Action Output

#	Time	Action	Message
1	21:18:27	select * from users	3990 row(s) returned

Del mismo modo que el anterior, para cargar los usuarios americanos se utilizara el csv american_users.csv y en el set se especificara que estos usuarios pertenecen a USA en el campo user_continent.

```
load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Sprint 04/american_users.csv'
into table users
fields terminated by ','
enclosed by '"'
lines terminated by '\n'
ignore 1 lines
(id, name, surname, phone, email, birth_date, country, city, postal_code, address)
set user_continent = 'USA';
```

Output

Action Output

#	Time	Action	Message
1	21:18:27	select * from users	3990 row(s) returned
2	21:21:09	load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Sp...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0

Se verifica que los datos del csv american_users.csv se han introducido correctamente en la tabla users.

```
select *
from users;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	name	surname	phone	email	birth_date	country	city	postal_code	address	user_continent
1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	Nov 17, 1985	United States	New York	10001	348-7818 Sagittis St.	USA
10	Robert	Mccarthy	(324) 746-6771	fermentum@protonmail.com	Apr 30, 1984	United States	San Jose	95101	P.O. Box 773	USA
100	Melodie	Mclean	1-677-221-7152	risus.varius@google.ca	Sep 15, 1989	United States	San Jose	95101	Ap #644-8492 Sagittis St.	USA
1000	Amkjrv	Qbulrxbp	+48-258-9936	amkjrv.qbulrxbp@example.com	May 17, 1970	Germany	Stuttgart	70173	215 Qbulrxbp St	EU
1001	Nfvrlb	Oydaiwbg	+94-121-2522	nfvrlb.oydaiwbg@example.com	Mar 4, 1994	Germany	Cologne	50667	121 Oydaiwbg St	EU
1002	Ijbfmd	Jbddzhvp	+70-120-3668	ijbfmd.jbddzhvp@example.com	Sep 27, 2001	Germany	Munich	80331	412 Jbddzhvp St	EU
1003	Uyciig	Sfbdymzj	+58-123-6968	uyciig.sfbdymzj@example.com	Jan 20, 1981	Germany	Stuttgart	70173	735 Sfbdymzj St	EU
1004	Yjqurq	Ojizvgqi	+77-944-2340	yjqurq.ojizvgqi@example.com	Jul 27, 1954	Germany	Munich	80331	685 Ojizvgqi St	EU

users 12

Output

Action Output

#	Time	Action	Message
2	21:21:09	load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0
3	21:21:41	select * from users	5000 row(s) returned

Para cargar los datos de products.csv será igual que el resto, solo será necesario especificar el csv y la tabla donde se insertaran los datos

```
load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Sprint 04/products.csv'
into table products
fields terminated by ','
enclosed by '"'
lines terminated by '\n'
ignore 1 lines
(id, product_name, price, color, weight, warehouse_id);
```

Output

Action Output

#	Time	Action	Message
1	21:23:24	load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Spri...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Se verifica que los datos del csv products.cs se han introducido correctamente en la tabla products.

```
select *
from products;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	product_name	Price	color	weight	warehouse_id
1	Direwolf Stannis	$161.11	#7c7c7c	1	WH-4
10	Karstark Dorne	$119.52	#f4f4f4	2.4	WH--5
100	south duel	$40.43	#6d6d6d	3	WH--95
11	Karstark Dorne	$49.70	#141414	2.7	WH--6
12	duel Direwolf	$181.60	#a8a8a8	2.1	WH--7
13	palpatine chewbacca	$139.59	#2b2b2b	1	WH--8
14	Direwolf	$147.53	#c4c4c4	2	WH--9
15	Stannis warden	$194.29	#dbdbdb	1.5	WH--10

products 13

Output

Action Output

#	Time	Action	Message
1	21:23:24	load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Sp...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0
2	21:23:45	select * from products	100 row(s) returned

Se verifica que los datos del csv credit_cards.csv se han introducido correctamente en la tabla credit_cards.

```
load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Sprint 04/credit_cards.csv'
into table credit_cards
fields terminated by ','
enclosed by '"'
lines terminated by '\n'
ignore 1 lines
(id, user_id, iban, pan, pin, cvv, track1, track2, expiring_date);
```

Output

Action Output

#	Time	Action	Message
1	17:11:27	load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Spri...	

Se elimina la columna user_id ya que como se ha especificado más arriba no sirve.

```
alter table credit_cards
drop column user_id;
```

Output

Action Output

#	Time	Action	Message
1	17:07:12	load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Spri...	
2	17:07:18	alter table credit_cards drop column user_id	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

```
select *
from credit_cards;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	iban	pan	pin	cvv	track1	track2	expiring_date
CcS-4857	XX4857591835292505850771	2314242385113924	1819	467	%B2314242385113924^LWCBUDLWCBUD^22...	%B2314242385113924=2410101518363164?	09/27/25
CcS-4858	XX8581768137002436094025	6582720299715533	3964	817	%B6582720299715533^TIQMVITIQMVI^2404...	%B6582720299715533=2411101104546272?	12/28/28
CcS-4859	XX7826930491423553609370	8861684536289642	4983	277	%B8861684536289642^COFBGDCOFBGD^280...	%B8861684536289642=25021017616653717	11/26/26
CcS-4860	XX5559590368835304645299	2481155515498459	6876	661	%B2481155515498459^TIUJTUTIUJTU^31040...	%B2481155515498459=26021015144143957	07/27/27
CcS-4861	XX2035182877195191627307	1308930301149557	5710	398	%B1308930301149557^HPOBNZHPOBNZ^330...	%B1308930301149557=28051017513050287	04/25/26

credit_cards 31

Output

Action Output

#	Time	Action	Message
1	17:12:16	alter table credit_cards drop column user_id	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
2	17:13:08	select * from credit_cards	5000 row(s) returned

A la hora de cargar los datos de transactions.csv surge el problema de que el campo product_ids contiene algunos registros que no solo tienen un id, si no varios id, esto causa un problema y es que al ser varios id juntos la clave foránea no funciona, ya que solo puede apuntar a un id, no a varios, para solucionar este problema se puede hacer de 2 formas, la primera sería crear una tabla intermedia con los id separados, esta forma rompería el modelo en estrella y la otra es convertir el campo product_ids en un json, pero no es recomendable porque relentiza las consultas, por ello, en este caso, se utilizara la primera forma, para ello, primero será necesario eliminar la clave foránea de product_ids que hace referencia a la tabla products, luego, se cargaran los datos en la tabla transaction y se creara la tabla intermedia llamada products_transaction, con los campos transactions_id y products_id, ambos serán clave primaria y clave foránea al mismo tiempo

```
166 • alter table transactions
167   drop foreign key fk_products;
168
```

Output

Action Output

#	Time	Action	Message
5	22:05:45	select * from transactions	0 row(s) returned
6	22:19:06	alter table transactions drop foreign key fk_products	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

```
175 • load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Sprint 04/transactions.csv'
176   into table transactions
177   fields terminated by ';'
178   enclosed by '"'
179   lines terminated by '\n'
180   ignore 1 lines
181   (id, card_id, business_id, timestamp, amount, declined, product_ids, user_id, lat, longitude);
182
```

Output

Action Output

#	Time	Action	Message
1	22:27:31	load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/Sp...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0

```
183 • select *
184   from transactions;
185
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
00043A49-2949-494B-A5DD-A5BAE3BB19DD	CcS-9294	b-2458	28/08/2024 7:16	395.43	0	16, 26, 97, 87	4713	4.619.992.926.158.270	14.355.402.821.327.600
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	21/12/2016 20:07	155.63	0	66, 69, 87	438	4.159.720.554.463.740	1.222.175.994.259.360
00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	14/07/2020 15:37	326.01	0	30, 11, 16, 81	2118	29.757.295.899.964.300	-9.537.963.676.492.450
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	04/09/2017 19:44	161.6	0	72	2115	5.354.888.376.797.020	-11.350.305.274.646.500
00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	05/01/2017 18:19	148.91	0	18	3025	5.220.836.951.654.170	5.690.806.474.241.330
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CcU-3358	b-2598	23/09/2023 4:51	294.59	0	35, 33, 19	215	53.553.485.560.256.000	-1.134.991.339.874.780
0009A151-9BCF-4E31-9053-A468FF77FAAB	CcS-7509	b-2546	31/12/2023 0:06	383.63	0	93, 55, 28, 91	2928	5.193.615.735.576.970	5.342.650.184.655.180
0009D494-6245-4DF9-955D-2C084191CFFB	CcS-8483	b-2526	18/07/2017 7:52	197.8	0	55. 8. 72	3902	4.549.200.875.032.750	-7.357.063.686.984.950

transactions 2 ×

Output

Action Output

#	Time	Action	Message
1	22:27:31	load data local infile 'E:/informacion y documentos/Curso analisis de datos IT Academy/MYSQL/Especializacion/S...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0
2	22:28:20	select * from transactions	100000 row(s) returned

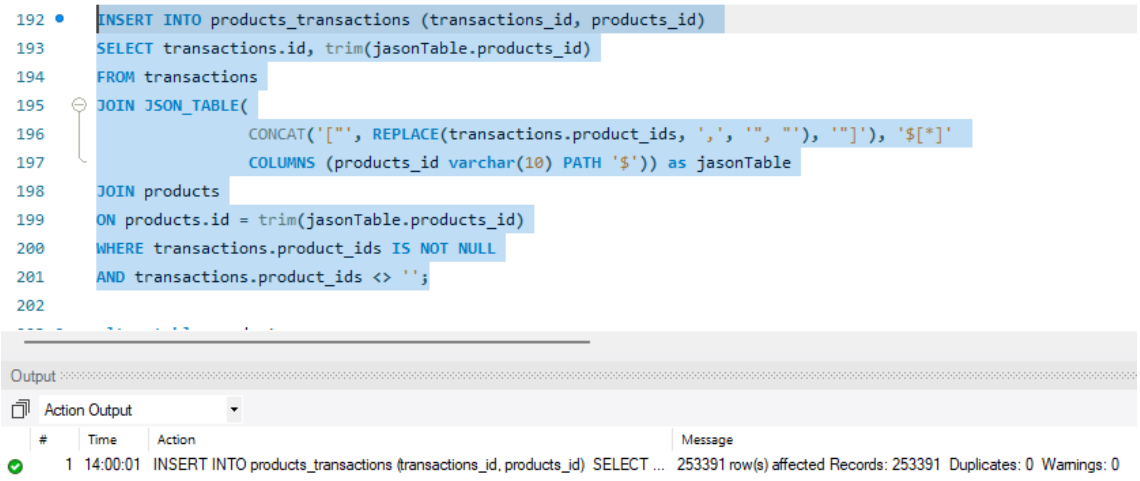
```
179 • create table products_transactions (
180       transactions_id varchar(255),
181       products_id varchar(255),
182       primary key (transactions_id, products_id),
183
184       constraint fk_transactions
185       foreign key (transactions_id)
186       references transactions(id),
187
188       constraint fk_products
189       foreign key (products_id)
190       references products(id));
191
```

Output

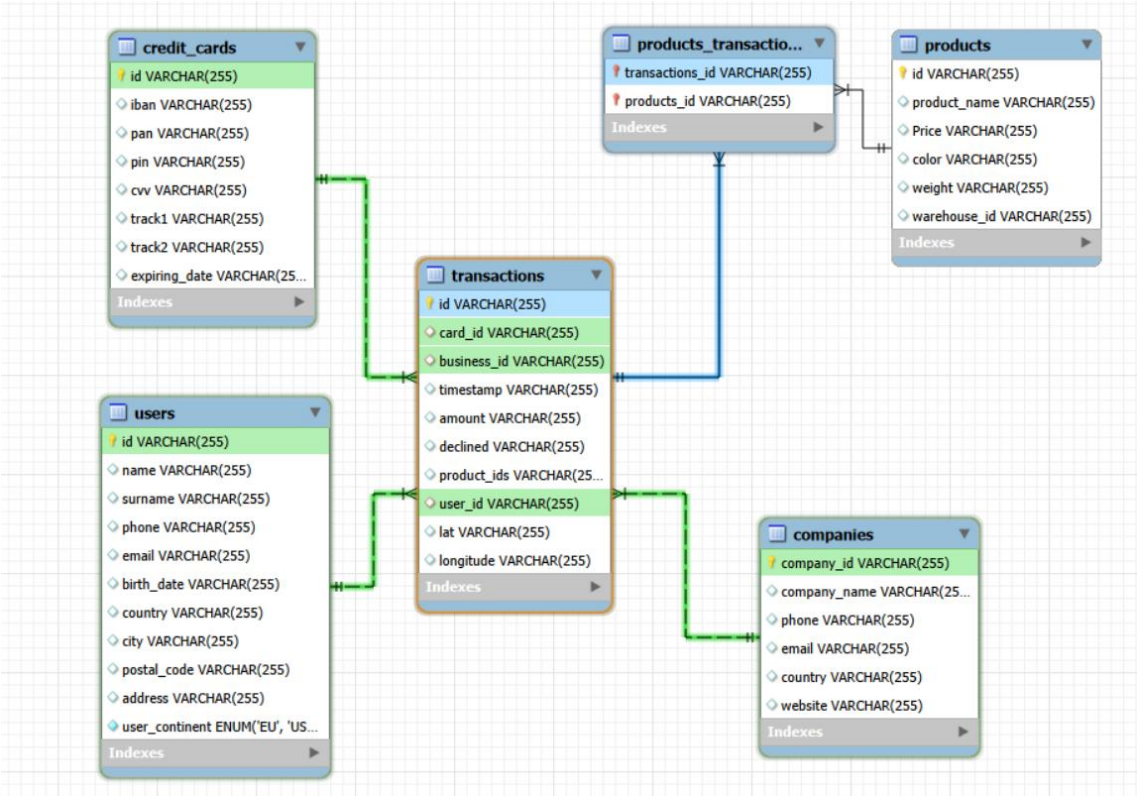
Action Output

#	Time	Action	Message
1	22:33:15	create table products_transactions (transactions_id var...	0 row(s) affected

Se insertan los id de transaction y product_ids en la nueva tabla products_transaction, para ello se utiliza una query en la que se selecciona el campo id de transaction, y products_id de la tabla temporal creada con json, se le aplica trim para eliminar espacios en blanco, se hace un join con la tabla json_table, esta convierte los registros (por ejemplo “100, 154, 262”) en un array json valido (“100”, “154”, “262”), concat cierra el array dando como resultado [“100”, “154”, “262”], ‘$[*]’ hace que se aplique a todos los elementos del array, con columns se define la columna products_id de tipo varchar con una longitud de 10 caracteres, y path ‘$’ hace que se aplique al valor actuar del elemento, es decir, a cada id del array, se le da un alias a la tabla temporal, se hace otro join a la tabla products para que coincidan los id de la tabla temporal con la tabla products, se aplica un filtro para evitar datos nulos y otro filtro para eliminar datos vacios.



Al revisar los datos he observado varios fallos y he tenido que modificar los campos de product_transactions después de realizar todo el sprint, en el archivo sql, en el código de esta parte los campos no aparecen como varchar(255) si no como varchar(36) transacrions_id y como varchar(10) products_id, porque se ha corregido después de modificar la longitud de los campos, el resto sigue igual.



Una vez creadas las tablas e importados los datos se procederá a cambiar los tipos de datos por los correctos, ya que todos están en varchar(255), para ello, primero será necesario verificar la longitud máxima de la columna con max(length(nombre_campo)), y luego se modifica con ese máximo y el tipo correcto de dato, como en el sprint 3, en un entorno real esto no es válido ya que los datos son variables y si entra un dato más grande de lo establecido causara problemas.

Tabla **companies**:

Table: companies

Columns:

company_id	varchar(255) PK
company_name	varchar(255)
phone	varchar(255)
email	varchar(255)
country	varchar(255)
website	varchar(255)

```
208 •  select max(length(company_id)) as idCompañia from companies;
209
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

idCompañia
6

Result 5

Output

Action Output

#	Time	Action	Message
1	10:36:05	select max(length(company_id)) as idCompañia from companies	1 row(s) returned

```
210 •  select max(length(company_name)) as nombreCompañia from companies;
211
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nombreCompañia
35

Result 6

Output

Action Output

#	Time	Action	Message
1	10:36:05	select max(length(company_id)) as idCompañia from companies	1 row(s) returned
2	10:36:21	select max(length(company_name)) as nombreCompañia from companies	1 row(s) returned

```
212 •  select max(length(phone)) as telefonoCompañia from companies;
213
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

telefonoCompañia
14

Result 7

Output

Action Output

#	Time	Action	Message
2	10:36:21	select max(length(company_name)) as nombreCompañia from companies	1 row(s) returned
3	10:36:37	select max(length(phone)) as telefonoCompañia from companies	1 row(s) returned

```
214 •  select max(length(email)) as emailCompañia from companies;
215
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emailCompañia
38

Result 8

Output

Action Output

#	Time	Action	Message
3	10:36:37	select max(length(phone)) as telefonoCompañia from companies	1 row(s) returned
4	10:36:55	select max(length(email)) as emailCompañia from companies	1 row(s) returned

```
216 •  select max(length(country)) as paisCompañia from companies;
217
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

paisCompañia
14

Result 9

Output

Action Output

#	Time	Action	Message
4	10:36:55	select max(length(email)) as emailCompañia from companies	1 row(s) returned
5	10:37:13	select max(length(country)) as paisCompañia from companies	1 row(s) returned

```
218 •  select max(length(website)) webCompañia from companies;
219
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

webCompañia
32

Result 10

Output

Action Output

#	Time	Action	Message
5	10:37:13	select max(length(country)) as paisCompañia from companies	1 row(s) returned
6	10:37:30	select max(length(website)) webCompañia from companies	1 row(s) returned

Una vez se hayan obtenido las longitudes máximas de los campos, al intentar modificarlos surgirá un error, y es debido a que no se puede modificar un valor con clave foránea por haberlos declarado con on delete cascade y on update cascade, por lo que hay que borrar todas las claves foráneas de product_transactions y transactions, modificar los valores y volver a definir las claves foráneas.

```
220 •  alter table companies
221    modify company_id int;
```

Output

Action Output

#	Time	Action	Message
6	10:37:30	select max(length(website)) webCompañia from companies	1 row(s) returned
7	10:40:11	alter table companies modify company_id int	Error Code: 3780. Referencing column 'business_id' and referenced column 'company_id' in foreign key con

```
229 •  alter table products_transactions
230    drop foreign key fk_transactions;
```

Output

Action Output

#	Time	Action	Message
2	10:51:18	alter table products_transactions drop foreign key fk_transactions	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

```
232 •  alter table products_transactions
233    drop foreign key fk_products;
```

Output

Action Output

#	Time	Action	Message
3	10:51:56	alter table products_transactions drop foreign key fk_products	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

```
229 •  alter table transactions
230    drop foreign key fk_credit_card;
```

Output

Action Output

#	Time	Action	Message
1	11:17:15	alter table transactions drop foreign key fk_credit_card	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

```
232 • alter table transactions
233   drop foreign key fk_companies;
234
```

Output

Action Output

#	Time	Action	Message
1	11:17:15	alter table transactions drop foreign key fk_credit_card	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
2	11:17:35	alter table transactions drop foreign key fk_companies	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

```
238 • alter table transactions
239   drop foreign key fk_users_transactions;
240
```

Output

Action Output

#	Time	Action	Message
1	11:18:14	alter table transactions drop foreign key fk_users_transactions	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

En el caso del id, la longitud máxima que ha resultado ha sido 6 caracteres, si en el futuro se ingresan más compañías el id seguirá aumentando con la posibilidad de superar esa longitud, por lo que se redefine con 8 caracteres para tener un poco de margen en caso de que el id aumente de 6 caracteres.

```
245 • alter table companies
246   modify company_id varchar(8);
247
```

Output

Action Output

#	Time	Action	Message
1	13:35:55	alter table companies modify company_id varchar(8)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0

En el campo Company_name sucede lo mismo que el anterior, al ser un dato variable en vez de poner 35 caracteres que ha salido como resultado, se pondrán 40 por si se añade un nombre más largo de 35 caracteres.

```
248 • alter table companies
249   modify company_name varchar(40);
250
```

Output

Action Output

#	Time	Action	Message
1	13:35:55	alter table companies modify company_id varchar(8)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0
2	13:41:25	alter table companies modify company_name varchar(40)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0

Al revisar el campo phone se puede observar que está compuesto solo de números, al no tener ninguna letra ni símbolo se podría poner el dato como int, pero aparte de tener espacios, si comienza por 0, el 0 desaparecería, algunos teléfonos tienen + al comienzo, no todos tienen la misma longitud, puede ser más largo que un int y además no se realizan operaciones matemáticas con ellos, asique se pondrá como varchar(14) que es lo longitud máxima del dato.

```
251 • alter table companies
252   modify phone varchar(14);
253
```

Output

Action Output

#	Time	Action	Message
1	13:48:18	alter table companies modify phone varchar(14)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0

El campo email por seguridad se le asigna una longitud de 100 porque hay correos muy largos.

```
254 • alter table companies
255   modify email varchar(100);
256
```

Output

Action Output

#	Time	Action	Message
1	13:48:18	alter table companies modify phone varchar(14)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0
2	13:57:59	alter table companies modify email varchar(100)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0

Al campo country se le asigna una longitud de 20, al comprobar la longitud da 14, se opta por poner 20 por si entra un país con más de 14 caracteres.

```
257 • alter table companies
258   modify country varchar(20);
259
```

Output

Action Output

#	Time	Action	Message
2	13:57:59	alter table companies modify email varchar(100)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0
3	13:58:37	alter table companies modify country varchar(20)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0

El campo website da una longitud de 32 caracteres, en su lugar se le asignan 50 por si entra alguna web que supere la longitud obtenida.

```
260 • alter table companies
261   modify website varchar(50);
262
```

Output

Action Output

#	Time	Action	Message
3	13:58:37	alter table companies modify country varchar(20)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0
4	13:59:02	alter table companies modify website varchar(50)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0

La tabla companies quedaría de la siguiente manera:

Table: companies

Columns:

company_id	varchar(8) PK
company_name	varchar(40)
phone	varchar(14)
email	varchar(100)
country	varchar(20)
website	varchar(50)

Tabla **credit_cards**:

Table: credit_cards

Columns:

id	varchar(255) PK
iban	varchar(255)
pan	varchar(255)
pin	varchar(255)
cvv	varchar(255)
track1	varchar(255)
track2	varchar(255)
expiring_date	varchar(255)

```
264 •  select max(length(id)) as id from credit_cards;
265
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id
8

Result 29

Output

Action Output

#	Time	Action	Message
1	18:06:58	select max(length(id)) as id from credit_cards	1 row(s) returned

```
266 •  select max(length(iban)) as iban from credit_cards;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban
31

Result 30

Output

Action Output

#	Time	Action	Message
1	18:06:58	select max(length(id)) as id from credit_cards	1 row(s) returned
2	18:07:50	select max(length(iban)) as iban from credit_cards	1 row(s) returned

```
268 • select max(length(pan)) as pan from credit_cards;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

pan
19

Result 25

Output

Action Output

#	Time	Action	Message
2	18:04:20	select max(length(iban)) from credit_cards	1 row(s) returned
3	18:05:35	select max(length(pan)) as pan from credit_cards	1 row(s) returned

```
270 • select max(length(pin)) as pin from credit_cards;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

pin
4

Result 32

Output

Action Output

#	Time	Action	Message
3	18:09:03	select max(length(pan)) as pan from credit_cards	1 row(s) returned
4	18:09:37	select max(length(pin)) as pin from credit_cards	1 row(s) returned

```
272 • select max(length(cvv)) as cvv from credit_cards;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cvv
3

Result 33

Output

Action Output

#	Time	Action	Message
4	18:09:37	select max(length(pin)) as pin from credit_cards	1 row(s) returned
5	18:11:57	select max(length(cvv)) as cvv from credit_cards	1 row(s) returned

```
274 •  select max(length(track1)) as track1 from credit_cards;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

track1
52

Result 34

Output

Action Output

#	Time	Action	Message
5	18:11:57	select max(length(cvv)) as cvv from credit_cards	1 row(s) returned
6	18:12:48	select max(length(track1)) as track1 from credit_cards	1 row(s) returned

```
276 •  select max(length(track2)) as track2 from credit_cards;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

track2
36

Result 35

Output

Action Output

#	Time	Action	Message
6	18:12:48	select max(length(track1)) as track1 from credit_cards	1 row(s) returned
7	18:13:11	select max(length(track2)) as track2 from credit_cards	1 row(s) returned

```
278 •  select max(length(expiring_date)) as expiring_date from credit_cards;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

expiring_date
8

Result 37

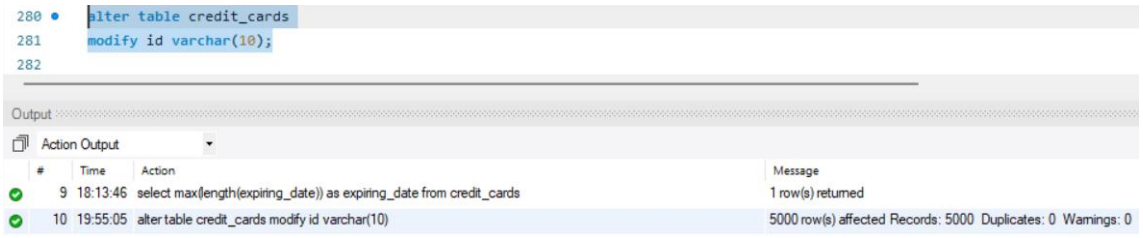
Output

Action Output

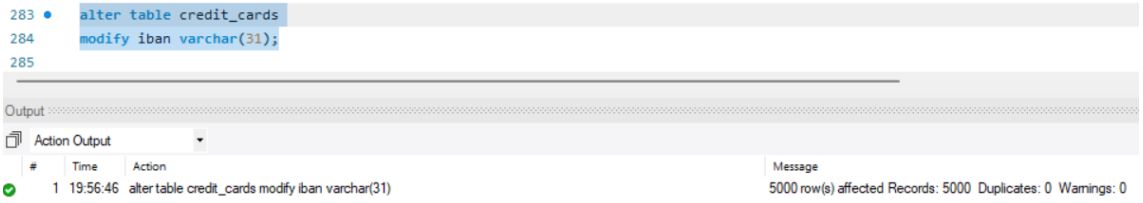
#	Time	Action	Message
8	18:13:33	select max(length(expiring_date)) as expiring_date from credit_cards	1 row(s) returned

Al igual que en el sprint 3, hay datos que en un entorno real no se guardarían, como el iban completo, el pin o el cvv, y con el campo expiring_date ocurre el mismo problema de la fecha al convertir el campo en tipo date.

Para la columna id se le asigna una longitud de 10 caracteres por si aumenta y supera los 8 iniciales.

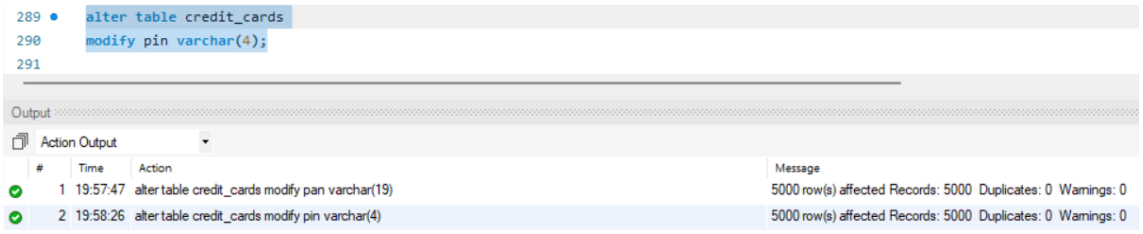


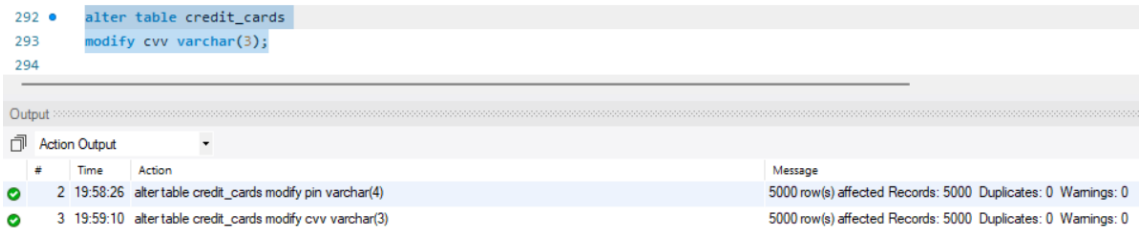
La columna iban se queda igual con la longitud obtenida de 31 caracteres.



La columna pan también se queda igual que la longitud obtenida.

Las columnas pin, cvv, track1 y track2 también se quedan con la longitud obtenida.





```
295 •  alter table credit_cards
296    modify track1 varchar(52);
297
```

Output

Action Output

#	Time	Action	Message
3	19:59:10	alter table credit_cards modify cvv varchar(3)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
4	20:02:06	alter table credit_cards modify track1 varchar(52)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
298 •  alter table credit_cards
299    modify track2 varchar(36);
300
```

Output

Action Output

#	Time	Action	Message
4	20:02:06	alter table credit_cards modify track1 varchar(52)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
5	20:02:36	alter table credit_cards modify track2 varchar(36)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

Con el campo expiring_date ocurre lo mismo que en el sprint 3, será necesario convertirlo a date, al observar los datos se puede ver que la fecha está en formato MM/DD/YY, el formato de date es DD/MM/YYYY o MM/DD/YYYY, al tener dos dígitos la fecha, si los datos son muy antiguos podría causar ambigüedad, ya que sql no sabe diferenciar si por ejemplo 15 seria 1915 o 2015, normalmente se suele guardar con el último día del mes, ya que las tarjetas de crédito no caducan un día concreto sino en un mes concreto, normalmente al final del mes que marque la tarjeta, para realizar esto sería necesario actualizar el tipo de dato a date, poner el ultimo día con last_day y str_to_date para la conversión del dato de varchar a date, limit 6000 se pone porque si no salta el modo seguro de mysql a la hora de realizar update, aunque con 5000 (que son los registros que tiene la tabla) sería suficiente.

```
320 •  UPDATE credit_cards
321    SET credit_cards.expiring_date =
322        LAST_DAY(
323            STR_TO_DATE(credit_cards.expiring_date, '%m/%d/%y')
324        )
325    limit 6000;
326
```

Output

Action Output

#	Time	Action	Message
1	12:56:54	UPDATE credit_cards SET credit_cards.expiring_date = LAST_DAY(STR_TO_DATE(credit_cards.ex...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0

```
327 •  alter table credit_cards
328    modify expiring_date date;
329
```

Output

Action Output

#	Time	Action	Message
1	12:56:54	UPDATE credit_cards SET credit_cards.expiring_date = LAST_DAY(STR_TO_DATE(credit_cards.ex...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0
2	12:57:57	alter table credit_cards modify expiring_date date	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

Table: credit_cards

Columns:

id	varchar(10) PK
iban	varchar(31)
pan	varchar(19)
pin	varchar(4)
cvv	varchar(3)
track1	varchar(52)
track2	varchar(36)
expiring_date	date

Tabla **products**:

Table: products

Columns:

id	varchar(255) PK
product_name	varchar(255)
Price	varchar(255)
color	varchar(255)
weight	varchar(255)
warehouse_id	varchar(255)

```
305 •  select max(length(id)) as id from products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id
3

Result 41

Output

Action Output

#	Time	Action	Message
1	20:27:02	select max(length(id)) as id from products	1 row(s) returned

```
307 •  select max(length(product_name)) as nombre_producto from products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nombre_producto
29

Result 42

Output

Action Output

#	Time	Action	Message
1	20:27:02	select max(length(id)) as id from products	1 row(s) returned
2	20:27:32	select max(length(product_name)) as nombre_producto from products	1 row(s) returned

```
309 •  select max(length(Price)) as precio from products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

precio
7

Result 43

Output

Action Output

#	Time	Action	Message
2	20:27:32	select max(length(product_name)) as nombre_producto from products	1 row(s) returned
3	20:28:03	select max(length(Price)) as precio from products	1 row(s) returned

```
311 •  select max(length(color)) as color from products;
312
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

color
7

Result 44

Output

Action Output

#	Time	Action	Message
3	20:28:03	select max(length(Price)) as precio from products	1 row(s) returned
4	20:29:30	select max(length(color)) as color from products	1 row(s) returned

```
313 •  select max(length(weight)) as peso from products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

peso
3

Result 45

Output

Action Output

#	Time	Action	Message
5	20:29:55	select * from products	100 row(s) returned
6	20:31:32	select max(length(weight)) as peso from products	1 row(s) returned

```
315 •  select max(length(warehouse_id)) as warehouse_id from products;
316
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouse_id
6

Result 46

Output

Action Output

#	Time	Action	Message
6	20:31:32	select max(length(weight)) as peso from products	1 row(s) returned
7	20:32:23	select max(length(warehouse_id)) as warehouse_id from products	1 row(s) returned

Se modifican los datos en base a las longitudes obtenidas

```
317 •  alter table products
318    modify id int;
319
```

Output

Action Output

#	Time	Action	Message
7	20:32:23	select max(length(warehouse_id)) as warehouse_id from products	1 row(s) returned
8	20:35:25	alter table products modify id int	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0

```
320 •  alter table products
321    modify product_name varchar(40);
322
```

Output

Action Output

#	Time	Action	Message
8	20:35:25	alter table products modify id int	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0
9	20:38:07	alter table products modify product_name varchar(40)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0

```
323 •  alter table products
324    rename column Price to price$;
325
```

Output

Action Output

#	Time	Action	Message
1	17:23:08	alter table products rename column Price to price$	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Se elimina el símbolo $ para tener solo los números y poder realizar operaciones matemáticas y se define el campo como decimal con dos decimales.

```
326 •  UPDATE products
327    SET price$ =
328        CAST(
329            REPLACE(price$, '$', '')
330            AS DECIMAL(10,2)
331        ) where id > 0;
332
```

Output

Action Output

#	Time	Action	Message
1	17:27:42	UPDATE products SET price$ = CAST(REPLACE(price$, '$', '') AS DECIMAL(10,2)) where ...	100 row(s) affected Rows matched: 100 Changed: 100 Warnings: 0

```
333 •  alter table products
334    modify price$ decimal(10,2);
```

Output

Action Output

#	Time	Action	Message
1	17:29:04	alter table products modify price$ decimal(10,2)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

```
326 •  alter table products
327    modify color varchar(7);
328
```

Output

Action Output

#	Time	Action	Message
1	20:42:21	alter table products modify color varchar(7)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0

```
332 •  alter table products
333    modify weight varchar(8);
334
```

Output

Action Output

#	Time	Action	Message
2	20:43:09	alter table products modify weight varchar(4)	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0
3	20:43:51	alter table products modify weight varchar(8)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Table: products

Columns:

id	int PK
product_name	varchar(40)
price$	decimal(10,2)
color	varchar(7)
weight	varchar(8)
warehouse_id	varchar(255)

Tabla **transactions**:

Table: transactions

Columns:

id	varchar(255) PK
card_id	varchar(255)
business_id	varchar(255)
timestamp	varchar(255)
amount	varchar(255)
declined	varchar(255)
product_ids	varchar(255)
user_id	varchar(255)
lat	varchar(255)
longitude	varchar(255)

```
336 •  select max(length(id)) as id from transactions;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id
36

Result 53

Output

Action Output

#	Time	Action	Message
1	20:59:14	select max(length(id)) as id from transactions	1 row(s) returned

```
338 •  select max(length(card_id)) as card_id from transactions;
339
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

card_id
8

Result 54

Output

Action Output

#	Time	Action	Message
1	20:59:14	select max(length(id)) as id from transactions	1 row(s) returned
2	20:59:36	select max(length(card_id)) as card_id from transactions	1 row(s) returned

```
340 •  select max(length(business_id)) as companies_id from transactions;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

companies_id
6

Result 56

Output

Action Output

#	Time	Action	Message
3	21:00:32	select max(length(business_id)) as companies_id from transactions	1 row(s) returned
4	21:00:40	select max(length(business_id)) as companies_id from transactions	1 row(s) returned

```
342 •  select max(length(timestamp)) as timestamp from transactions;
343
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

timestamp
16

Result 57

Output

Action Output

#	Time	Action	Message
4	21:00:40	select max(length(business_id)) as companies_id from transactions	1 row(s) returned
5	21:03:59	select max(length(timestamp)) as timestamp from transactions	1 row(s) returned

```
344 •  select max(length(amount)) as amount from transactions;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

amount
6

Result 58

Output

Action Output

#	Time	Action	Message
5	21:03:59	select max(length(timestamp)) as timestamp from transactions	1 row(s) returned
6	21:04:41	select max(length(amount)) as amount from transactions	1 row(s) returned

```
346 •  select max(length(declined)) as declinada from transactions;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

declinada
1

Result 59

Output

Action Output

#	Time	Action	Message
6	21:04:41	select max(length(amount)) as amount from transactions	1 row(s) returned
7	21:05:26	select max(length(declined)) as declinada from transactions	1 row(s) returned

```
348 •  select max(length(product_ids)) as products_id from transactions;
349
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

products_id
19

Result 60

Output

Action Output

#	Time	Action	Message
7	21:05:26	select max(length(declined)) as declinada from transactions	1 row(s) returned
8	21:06:12	select max(length(product_ids)) as products_id from transactions	1 row(s) returned

```
350 •  select max(length(user_id)) as user_id from transactions;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

user_id
4

Result 61

Output

Action Output

#	Time	Action	Message
8	21:06:12	select max(length(product_ids)) as products_id from transactions	1 row(s) returned
9	21:06:59	select max(length(user_id)) as user_id from transactions	1 row(s) returned

```
352 •  select max(length(lat)) as latitud from transactions;
353
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

latitud
22

Result 62

Output

Action Output

#	Time	Action	Message
9	21:06:59	select max(length(user_id)) as user_id from transactions	1 row(s) returned
10	21:07:33	select max(length(lat)) as latitud from transactions	1 row(s) returned

```
354 •  select max(length(longitude)) as longitud from transactions;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

longitud
24

Result 63

Output

Action Output

#	Time	Action	Message
10	21:07:33	select max(length(lat)) as latitud from transactions	1 row(s) returned
11	21:08:27	select max(length(longitude)) as longitud from transactions	1 row(s) returned

Se modifica la longitud de los datos en base a las longitudes onbtenidas.

```
356 •  alter table transactions
357    modify id varchar(36);
```

Output

Action Output

#	Time	Action	Message
12	21:09:22	select * from transactions	100000 row(s) returned
13	21:09:55	alter table transactions modify id varchar(36)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

```
359 •  alter table transactions
360    modify card_id varchar(10); -- igual que id en la tabla credit_cards
361
```

Output

Action Output

#	Time	Action	Message
1	21:12:33	alter table transactions modify card_id varchar(10)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

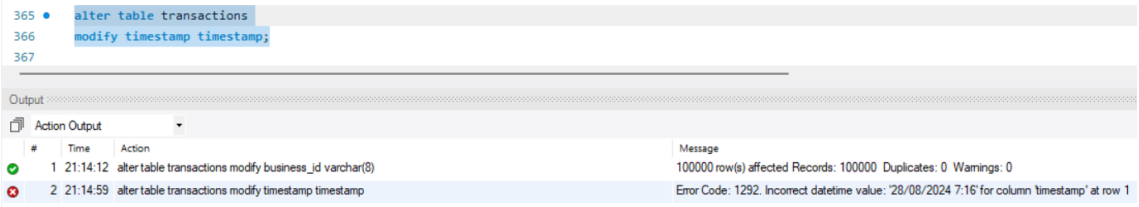
```
362 •  alter table transactions
363    modify business_id varchar(8); -- igual que id en la tabla companies
364
```

Output

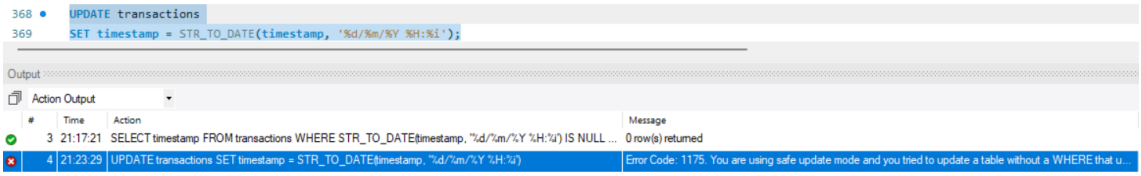
Action Output

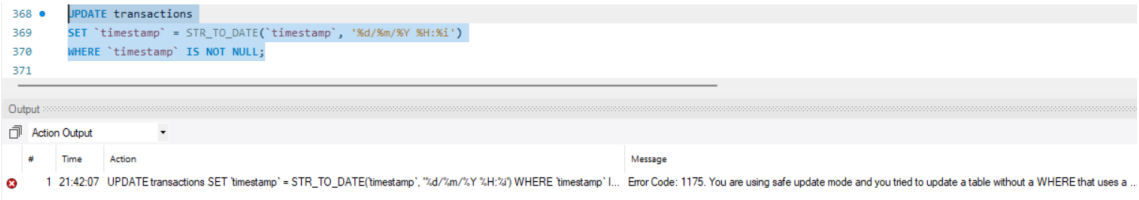
#	Time	Action	Message
1	21:14:12	alter table transactions modify business_id varchar(8)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

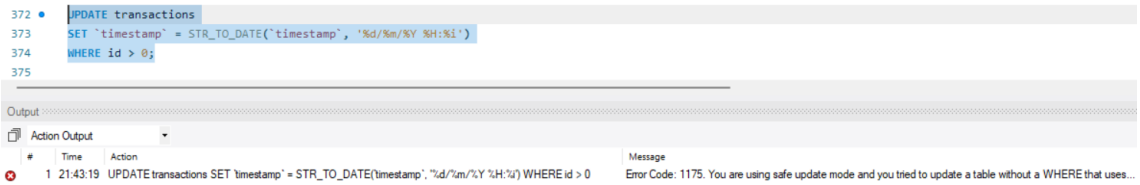
El campo timestamp no se puede convertir a timestamp directamente porque los datos del campo son incorrectos para el tipo de dato timestamp.

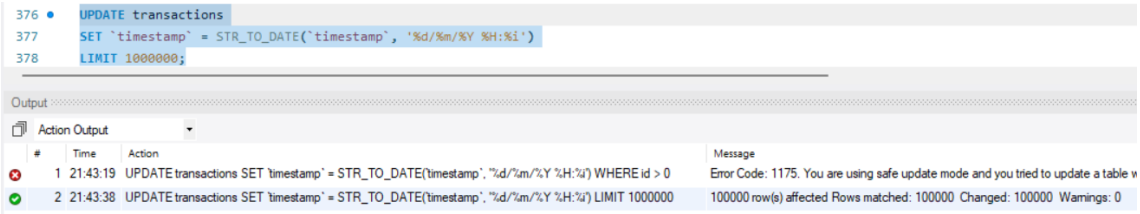


Al intentar convertirlo a timestamp con update da errorpor el modo seguro, no funciona ni con str_to_date, ni con un filtro where, solo funciona con un limit, que es igual de grande que el toral de los registros de la tabla transactions

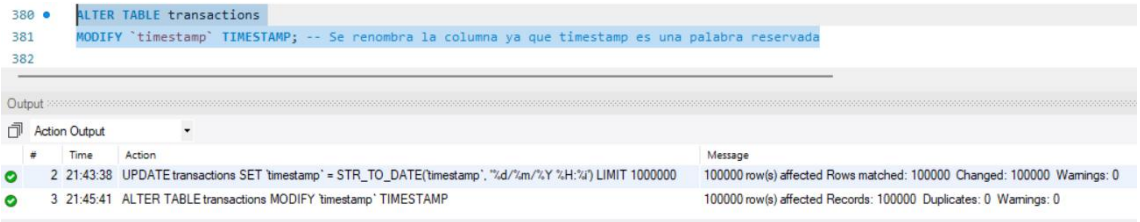








Se renombra la columna timestamp por TIMESTAMP ya que timestamp es una palabra reservada y puede dar lugar a errores.



```
383 • alter table transactions
384   modify amount decimal(10,2);
```

Output

Action Output

#	Time	Action	Message
1	17:00:26	alter table transactions modify amount decimal(10,2)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

```
386 • alter table transactions
387   modify declined tinyint;
388
```

Output

Action Output

#	Time	Action	Message
1	21:48:46	alter table transactions modify amount varchar(8)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
2	21:51:15	alter table transactions modify declined tinyint	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

```
392 • alter table transactions
393   modify user_id int;
394
```

Output

Action Output

#	Time	Action	Message
5	21:55:28	select * from users	5000 row(s) returned
6	21:55:58	alter table transactions modify user_id int	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

```
395 • alter table transactions
396   modify lat varchar(22);
397
```

Output

Action Output

#	Time	Action	Message
6	21:55:58	alter table transactions modify user_id int	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
7	21:58:03	alter table transactions modify lat varchar(22)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

```
398 • alter table transactions
399   modify longitude varchar(24);
400
```

Output

Action Output

#	Time	Action	Message
9	21:59:07	alter table transactions modify lat varchar(22)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
10	21:59:19	alter table transactions modify longitude varchar(24)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Table: transactions

Columns:

id	varchar(36) PK
card_id	varchar(10)
business_id	varchar(8)
timestamp	timestamp
amount	varchar(8)
declined	tinyint
product_ids	varchar(255)
user_id	int
lat	varchar(22)
longitude	varchar(24)

Tabla **users**:

Table: users

Columns:

id	varchar(255) PK
name	varchar(255)
surname	varchar(255)
phone	varchar(255)
email	varchar(255)
birth_date	varchar(255)
country	varchar(255)
city	varchar(255)
postal_code	varchar(255)
address	varchar(255)
user_continent	enum('EU','USA')

```
402 •  select max(length(id)) as id from users;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id
4

Result 64

Output

Action Output

#	Time	Action	Message
10	21:59:19	alter table transactions modify longitude varchar(24)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
11	01:10:03	select max(length(id)) as id from users	1 row(s) returned

```
404 •  select max(length(name)) as nombre from users;
405
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nombre
9

Result 65

Output

Action Output

#	Time	Action	Message
11	01:10:03	select max(length(id)) as id from users	1 row(s) returned
12	01:15:37	select max(length(name)) as nombre from users	1 row(s) returned

```
406 •  select max(length(surname)) as apellido from users;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

apellido
11

Result 66

Output

Action Output

#	Time	Action	Message
12	01:15:37	select max(length(name)) as nombre from users	1 row(s) returned
13	01:16:29	select max(length(surname)) as apellido from users	1 row(s) returned

```
408 •  select max(length(phone)) as telefono from users;
409
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

telefono
15

Result 67

Output

Action Output

#	Time	Action	Message
13	01:16:29	select max(length(surname)) as apellido from users	1 row(s) returned
14	01:17:10	select max(length(phone)) as telefono from users	1 row(s) returned

```
410 •  select max(length(email)) as email from users;
411
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

email
40

Result 68

Output

Action Output

#	Time	Action	Message
14	01:17:10	select max(length(phone)) as telefono from users	1 row(s) returned
15	01:18:28	select max(length(email)) as email from users	1 row(s) returned

```
412 •  select max(length(birth_date)) as birth_date from users;
413
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

birth_date
12

Result 69

Output

Action Output

#	Time	Action	Message
15	01:18:28	select max(length(email)) as email from users	1 row(s) returned
16	01:49:07	select max(length(birth_date)) as birth_date from users	1 row(s) returned

```
414 •  select max(length(country)) as pais from users;
415
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

pais
14

Result 72

Output

Action Output

#	Time	Action	Message
1	01:51:39	select max(length(country)) as pais from users	1 row(s) returned

```
416 •  select max(length(city)) as ciudad from users;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ciudad
12

Result 73

Output

Action Output

#	Time	Action	Message
1	01:51:39	select max(length(country)) as pais from users	1 row(s) returned
2	01:52:39	select max(length(city)) as ciudad from users	1 row(s) returned

```
418 •  select max(length(postal_code)) as codigo_postal from users;
419
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

codigo_postal
8

Result 74

Output

Action Output

#	Time	Action	Message
2	01:52:39	select max(length(city)) as ciudad from users	1 row(s) returned
3	01:54:07	select max(length(postal_code)) as codigo_postal from users	1 row(s) returned

```
420 •  select max(length(address)) as direccion from users;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

direccion
33

Result 75

Output

Action Output

#	Time	Action	Message
3	01:54:07	select max(length(postal_code)) as codigo_postal from users	1 row(s) returned
4	01:55:25	select max(length(address)) as direccion from users	1 row(s) returned

```
422 •  select max(length(user_continent)) as continente_usuario from users;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

continente_usuario
3

Result 77

Output

Action Output

#	Time	Action	Message
5	01:56:33	select max(length(user_continent)) as cointinente_usuario from users	1 row(s) returned
6	01:56:52	select max(length(user_continent)) as continente_usuario from users	1 row(s) returned

Se modifican los tipos de datos y las longitudes

```
424 •  alter table users
425    modify id int; -- Igual que en tranasctions
426
```

Output

Action Output

#	Time	Action	Message
1	10:51:26	alter table users modify id int	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
427 •  alter table users
428    modify name varchar(12);
429
```

Output

Action Output

#	Time	Action	Message
1	10:51:26	alter table users modify id int	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
2	10:52:18	alter table users modify name varchar(12)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
430 •  alter table users
431    modify surname varchar(12);
432
```

Output

Action Output

#	Time	Action	Message
2	10:52:18	alter table users modify name varchar(12)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
3	10:52:56	alter table users modify surname varchar(12)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
433 •  alter table users
434    modify phone varchar(16);
435
```

Output

Action Output

#	Time	Action	Message
3	10:52:56	alter table users modify surname varchar(12)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
4	10:53:38	alter table users modify phone varchar(16)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
436 •  alter table users
437    modify email varchar(50);
438
```

Output

Action Output

#	Time	Action	Message
4	10:53:38	alter table users modify phone varchar(16)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
5	10:54:21	alter table users modify email varchar(50)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
439 •  alter table users
440    modify birth_date date;
441
```

Output

Action Output

#	Time	Action	Message
1	11:00:03	alter table users modify birth_date date	Error Code: 1292. Incorrect date value: 'Nov 17, 1985' for column 'birth_date' at row 1

Esto solo funcionará si la abreviatura esta en ingles

```
442 •  UPDATE users
443    SET birth_date = STR_TO_DATE(birth_date, '%b %d, %Y');
444
```

Output

Action Output

#	Time	Action	Message
2	11:00:45	UPDATE users SET birth_date = STR_TO_DATE(birth_date, '%b %d, %Y')	Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE tha...

```
442 •  UPDATE users
443    SET birth_date = STR_TO_DATE(birth_date, '%b %d, %Y')
444    limit 6000;
445
```

Output

Action Output

#	Time	Action	Message
1	11:01:52	UPDATE users SET birth_date = STR_TO_DATE(birth_date, '%b %d, %Y') limit 6000	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0

```
446 •  alter table users
447    modify birth_date date;
448
```

Output

Action Output

#	Time	Action	Message
1	11:01:52	UPDATE users SET birth_date = STR_TO_DATE(birth_date, '%b %d, %Y') limit 6000	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0
2	11:02:15	alter table users modify birth_date date	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
449 •  alter table users
450    modify country varchar(16);
451
```

Output

Action Output

#	Time	Action	Message
2	11:02:15	alter table users modify birth_date date	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
3	11:03:02	alter table users modify country varchar(16)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
455 •  alter table users
456    modify city varchar(16);
457
```

Output

Action Output

#	Time	Action	Message
3	11:03:02	alter table users modify country varchar(16)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
4	11:04:41	alter table users modify city varchar(16)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
458 •  alter table users
459    modify postal_code varchar(8);
460
```

Output

Action Output

#	Time	Action	Message
4	11:04:41	alter table users modify city varchar(16)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
5	11:05:31	alter table users modify postal_code varchar(8)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
461 •  alter table users
462    modify address varchar(40);
463
```

Output

Action Output

#	Time	Action	Message
5	11:05:31	alter table users modify postal_code varchar(8)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
6	11:06:06	alter table users modify address varchar(40)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
464 •  alter table users
465    modify user_continent varchar(3);
466
```

Output

Action Output

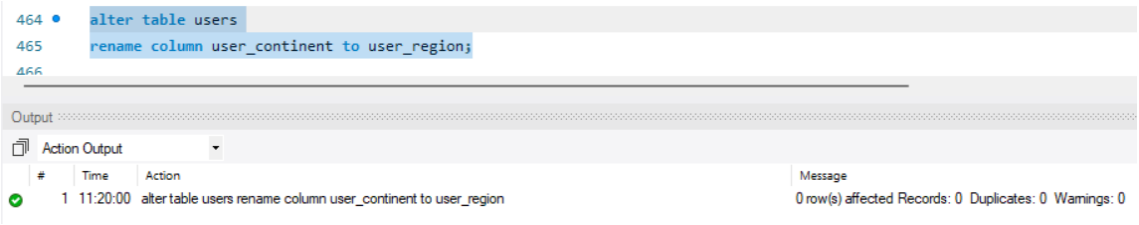
#	Time	Action	Message
6	11:06:06	alter table users modify address varchar(40)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
7	11:06:46	alter table users modify user_continent varchar(3)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

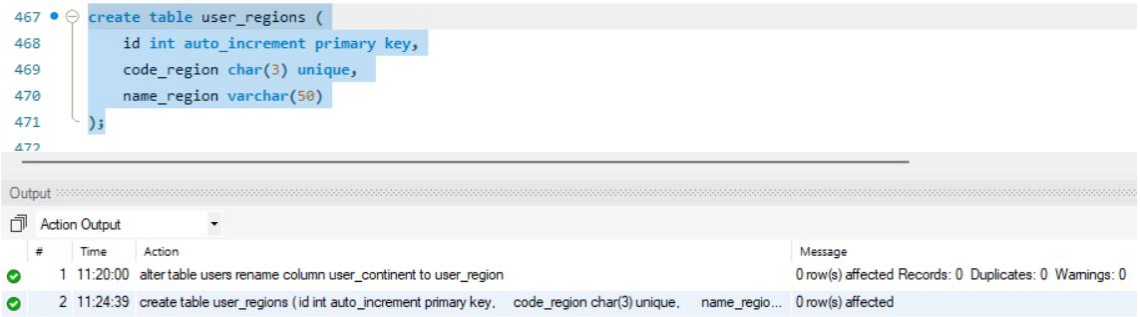
Table: users

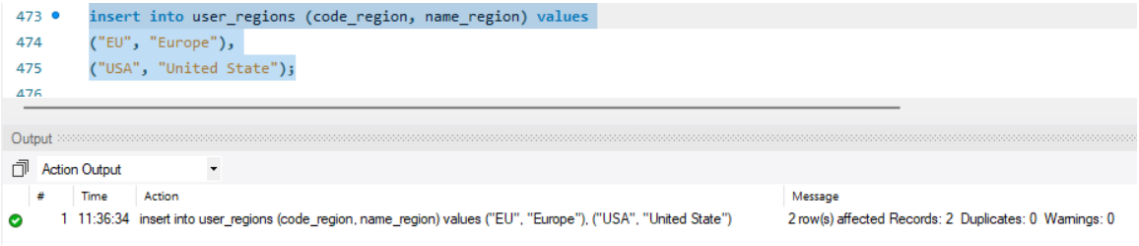
Columns:

id	int PK
name	varchar(12)
surname	varchar(12)
phone	varchar(16)
email	varchar(50)
birth_date	date
country	varchar(16)
city	varchar(16)
postal_code	varchar(8)
address	varchar(40)
user_region	varchar(3)

Para user continent se crea una tabla nueva para normalizar los datos al igual que los id entre transactions y products, de esta manera si en el futuro fuese necesario introducir mas regiones seria mas sencillo







```
477 • alter table users
478   add id_user_regions int;
479
```

Output

Action Output

#	Time	Action	Message
1	11:36:34	insert into user_regions (code_region, name_region) values ("EU", "Europe"), ("USA", "United State")	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
2	11:37:45	alter table users add id_user_regions int	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

```
480 • update users
481   join user_regions
482   on users.user_region = user_regions.code_region
483   set users.id_user_regions = user_regions.id
484   where users.id > 0;
485
```

Output

Action Output

#	Time	Action	Message
1	12:08:51	update users join user_regions on users.user_region = user_regions.code_region set users.id_user_regions =...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0

```
488 • alter table users
489   add constraint fk_id_user_regions
490   foreign key (id_user_regions)
491   references user_regions(id);
492
```

Output

Action Output

#	Time	Action	Message
3	12:10:37	SELECT COUNT(*) FROM users WHERE id_user_regions IS NULL	1 row(s) returned
4	12:11:30	alter table users add constraint fk_id_user_regions foreign key (id_user_regions) references user_regions(id)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
493 • SELECT *
494   FROM users
495   JOIN user_regions
496   ON users.id_user_regions = user_regions.id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

id	name	surname	phone	email	birth_date	country	city	postal_code	address	user_region	id_user_regions	id	code_region	name_region
151	Meghan	Hayden	0800 746 6747	arcu.vel@hotmail.ca	1980-07-02	United Kingdom	London	EC1A 1BB	Ap #432-4493 Aliquet Rd.	EU	1	1	EU	Europe
152	Hakeem	Alford	(0111) 367 0184	adipiscing.ligula@google.edu	1979-09-30	United Kingdom	Birmingham	B1 1AA	551-8930 Lobortis Street	EU	1	1	EU	Europe
153	Keegan	Pugh	(016977) 3851	sodales.nisi@aol.org	1994-07-27	United Kingdom	London	EC1A 1BB	Ap #312-5898 Consectetuer St.	EU	1	1	EU	Europe
154	Cooper	Bullock	(021) 2521 6627	et@outlook.net	1986-11-02	United Kingdom	Manchester	M1 1AE	872-1866 Pede Rd.	EU	1	1	EU	Europe
155	Joshua	Russell	055 4409 5286	justo.nec.ante@outlook.edu	1984-01-23	United Kingdom	Manchester	M1 1AE	Ap #285-4727 Auctor. Av.	EU	1	1	EU	Europe

Result 11

Output

Action Output

#	Time	Action	Message	Durati
1	16:25:44	SELECT * FROM users JOIN user_regions ON users.id_user_regions = user_regions.id	5000 row(s) returned	0.000

Tabla **product_transactions**:

Table: products_transactions

Columns:

transactions_id	varchar(255) PK
products_id	varchar(255) PK

```
499 • alter table products_transactions
500   modify transactions_id varchar(36);
501
```

Output

Action Output

#	Time	Action	Message
1	16:45:06	alter table products_transactions modify transactions_id varchar(36)	252572 row(s) affected Records: 252572 Duplicates: 0 Warnings: 0

```
502 • alter table products_transactions
503   modify products_id int;
504
```

Output

Action Output

#	Time	Action	Message
1	16:46:37	alter table products_transactions modify products_id int	252572 row(s) affected Records: 252572 Duplicates: 0 Warnings: 0

Al tener la relación ya creada no es necesaria la columna user_region, ya que saldría dos veces y es algo redundante, se elimina de la tabla users

```
505 • alter table users
506   drop column user_region;
507
```

Output

Action Output

#	Time	Action	Message
1	17:03:13	alter table users drop column user_region	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Table: users

Columns:

id	int PK
name	varchar(12)
surname	varchar(12)
phone	varchar(16)
email	varchar(50)
birth_date	date
country	varchar(16)
city	varchar(16)
postal_code	varchar(8)
address	varchar(40)
id_user_regions	int

Por último, se vuelven a definir las claves foráneas de transactions y products_transactions

```
498 • alter table transactions
499   add constraint fk_credit_card
500   foreign key (card_id)
501   references credit_cards(id)
502   on delete cascade
503   on update cascade;
504
```

Output

Action Output

#	Time	Action	Message
1	16:25:44	SELECT * FROM users JOIN user_regions ON users.id_user_regions = user_regions.id	5000 row(s) returned
2	16:28:32	alter table transactions add constraint fk_credit_card foreign key (card_id) references credit_cards(id) on delete cascade ...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

```
505 • alter table transactions
506   add constraint fk_companies
507   foreign key (business_id)
508   references companies(company_id)
509   on delete cascade
510   on update cascade;
511
```

Output

Action Output

#	Time	Action	Message
2	16:28:32	alter table transactions add constraint fk_credit_card foreign key (card_id) references credit_cards(id) on delete cascad...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
3	16:29:02	alter table transactions add constraint fk_companies foreign key (business_id) references companies(company_id) on d...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

```
512 • alter table transactions
513   add constraint fk_users_transactions
514   foreign key (user_id)
515   references users(id)
516   on delete cascade
517   on update cascade;
518
```

Output

Action Output

#	Time	Action	Message
1	16:31:11	alter table transactions add constraint fk_users_transactions foreign key (user_id) references users(id) on delete cascade o...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

```
519 • alter table products_transactions
520   add constraint fk_transactions
521   foreign key (transactions_id)
522   references transactions(id);
523
```

Output

Action Output

#	Time	Action	Message
1	16:31:11	alter table transactions add constraint fk_users_transactions foreign key (user_id) references users(id) on del...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
2	16:37:45	alter table products_transactions add constraint fk_transactions foreign key (transactions_id) references tran...	252572 row(s) affected Records: 252572 Duplicates: 0 Warnings: 0

```
528 • alter table products_transactions
529   add constraint fk_transactions
530   foreign key (transactions_id)
531   references transactions(id);
532
```

Output

Action Output

#	Time	Action	Message
1	16:48:11	alter table products_transactions add constraint fk_transactions foreign key (transactions_id) references tran...	252572 row(s) affected Records: 252572 Duplicates: 0 Warnings: 0

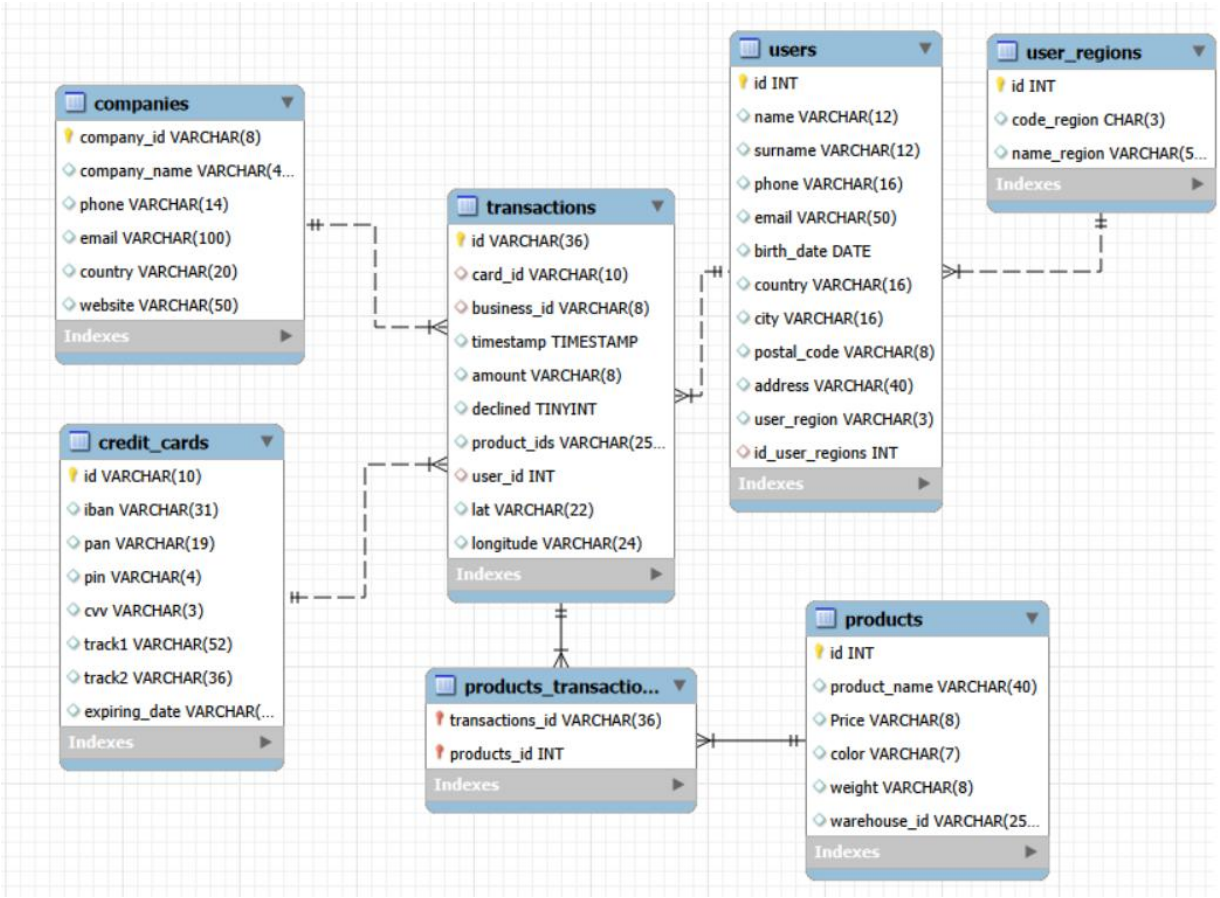
```
533 • alter table products_transactions
534   add constraint fk_products
535   foreign key (products_id)
536   references products(id);
537
```

Output

Action Output

#	Time	Action	Message
1	16:48:11	alter table products_transactions add constraint fk_transactions foreign key (transactions_id) references tran...	252572 row(s) affected Records: 252572 Duplicates: 0 Warnings: 0
2	16:49:28	alter table products_transactions add constraint fk_products foreign key (products_id) references products(id)	252572 row(s) affected Records: 252572 Duplicates: 0 Warnings: 0

Después de todos los cambios realizados el diagrama definitivo seria el siguiente:



En este diagrama se puede observar que todas las tablas están relacionadas con transactions de manera 1-N, es decir desde las tablas companies, credit_card, users y products (a través de products_transactions con business_id, card_id, user_id e id respectivamente) tienen una relación 1-N hacia transactions, la tabla user_region es 1-N de user_region hacia users, y esto es porque un usuario solo puede pertenecer a una región, pero en una región puede haber varios usuarios, en la tabla products_transactions se puede observar que los dos campos tienen una llave roja/naranja, mientras que el resto tiene una llave dorada, esto es porque ambos campos son clave primaria y clave foránea al mismo tiempo

Ejercicio 1

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

```
587 • SELECT *
588   FROM users u
589   where exists (
590       SELECT 1
591       FROM transactions t
592       where t.user_id = u.id and declined = 0
593       GROUP BY user_id
594       HAVING COUNT(id) > 80
595   );
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	name	surname	phone	email	birth_date	country	city	postal_code	address	id_user_regions
185	Molly	Gilliam	0800 120 8023	donec@outlook.couk	1993-12-21	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 Mi Rd.	1
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	1976-08-20	Germany	Stuttgart	70173	82 Hwcru Street	1
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	1974-05-03	Italy	Genoa	16100	53 Astuw Street	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

users 11

Output

Action Output

#	Time	Action	Message
1	12:57:55	SELECT * FROM users u where exists (SELECT 1 FROM transactions t where t.user_id = u.id and d...	3 row(s) returned

Ejercicio 2

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

Primero se verifica que la compañía Donec Ltd existe

```
560 • select *
561   from companies
562   where company_name = "Donec Ltd";
563
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

company_id	company_name	phone	email	country	website
b-2242	Donec Ltd	01 25 51 37 37	at.iaculis@hotmail.couk	Norway	https://nytimes.com/user/110
NULL	NULL	NULL	NULL	NULL	NULL

companies 19

Output

Action Output

#	Time	Action	Message
1	17:08:26	SELECT users.id AS userId, users.name, transactions2.conteoTransacciones FROM users JOIN (...	4 row(s) returned
2	17:09:18	select * from companies where company_name = "Donec Ltd"	1 row(s) returned

Una vez comprobado que existe se realiza la consulta

```sql
select c.company_id, c.company_name, cc.iban, round(avg(t.amount),2) as mediaVentas
from companies c
join transactions t
on c.company_id = t.business_id
join credit_cards cc
on t.card_id = cc.id
where company_name = "Donec Ltd" and declined = 0
group by c.company_id, c.company_name, cc.iban;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_id	company_name	iban	mediaVentas
b-2242	Donec Ltd	XX9114064011255863075868O5	356.25
b-2242	Donec Ltd	SK9446370242474562577506	142.96
b-2242	Donec Ltd	XX7767529178459529755556 40	257.37
b-2242	Donec Ltd	XX4138273622897193049089 90	139.59
b-2242	Donec Ltd	XX3477872460707696107803 08	240.41

Result 12

Output

Action Output

#	Time	Action	Message
1	12:57:55	SELECT * FROM users u where exists (SELECT 1 FROM transactions t where t.user_id = u.id and d...	3 row(s) returned
2	13:00:21	select c.company_id, c.company_name, cc.iban, round(avg(t.amount),2) as mediaVentas from companies c j...	370 row(s) returned

Nivel 2

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas entonces es inactivo, si al menos una no es rechazada entonces es activo. Partiendo de esta tabla responde:

```sql
create table credit_card_status (
    card_id varchar(10) primary key,
    status enum('ACTIVE', 'INACTIVE') not null,
    updated_at timestamp default current_timestamp
);
```

Output

Action Output

#	Time	Action	Message
1	11:29:37	create table credit_card_status (card_id varchar(10) primary key,...	0 row(s) affected

```sql
insert into credit_card_status (card_id, status)
select sub.card_id,
                            case
                                when SUM(sub.declined) = 3 then 'INACTIVE'
                                else 'ACTIVE'
                            end as status
from (select transactions.card_id, transactions.declined,
                                                        row_number() over (
                                                        partition by transactions.card_id
                                                        order by transactions.timestamp desc) as rowNumber
      from transactions) as sub
where rowNumber <= 3
group by sub.card_id
on duplicate key update
    status = values(status),
    updated_at = current_timestamp;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	11:37:10	insert into credit_card_status (card_id, status) select sub.card_id...	5000 row(s) affected, 1 warning(s): 1287 'VALUES function' is de...	0.438 sec

Esta query inserta los datos en la tabla credit_card_status de manera que el resultado es la visualización de cada tarjeta y si esta activa o no, esto funciona gracias a la subquery sub, que asigna un numero secuencial a cada fila con row_number, con partition by reinicia la numeración para cada tarjeta, con order by ordena la fecha de la más nueva a la más antigua, con esta subquery el case puede valorar si las 3 últimas transacciones han sido declinadas o no, si las 3 últimas transacciones fueron declinadas, en el campo status aparecerá la tarjeta como inactiva, si al menos una de las 3 últimas transacciones ha sido declinada, aparecerá la tarjeta como activa, where rowNumber <= 3 selecciona las 3 transacciones más recientes de cada tarjeta, se agrupa por el card_id de la subquery para poder hacer el sum() y on duplicate key update actualiza el status si aparece una tarjeta repetida para evitar errores y mantener la información actualizada.

```sql
select *
from credit_card_status;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

card_id	status	updated_at
CcS-4867	ACTIVE	2026-02-09 17:04:34
CcS-4868	ACTIVE	2026-02-09 17:04:34
CcS-4869	ACTIVE	2026-02-09 17:04:34
CcS-4870	INACTIVE	2026-02-09 17:04:34
CcS-4871	ACTIVE	2026-02-09 17:04:34
CcS-4872	ACTIVE	2026-02-09 17:04:34
CcS-4873	ACTIVE	2026-02-09 17:04:34
CcS-4874	ACTIVE	2026-02-09 17:04:34
CcS-4875	ACTIVE	2026-02-09 17:04:34
CcS-4876	ACTIVE	2026-02-09 17:04:34

credit_card_status 10

Output

Action Output

#	Time	Action	Message
1	17:05:02	select * from credit_card_status	5000 row(s) returned

Se declara la clave foránea de card_id de credit_card_status, ahora es clave primaria y foranea

```
641 •  alter table credit_card_status
642    add constraint fk_credit_card_id
643    foreign key (card_id)
644    references credit_cards(id)
645    on delete cascade
646    on update cascade;
```

Output

Action Output

#	Time	Action	Message
1	13:52:09	alter table credit_card_status add constraint fk_credit_card_id foreign key (card_id) references credit_cards(id)...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

Se declara unique el id para que no se repita

```
648 •  alter table credit_card_status
649    add constraint uq_credit_card_status_card
650    unique (card_id);
```

Output

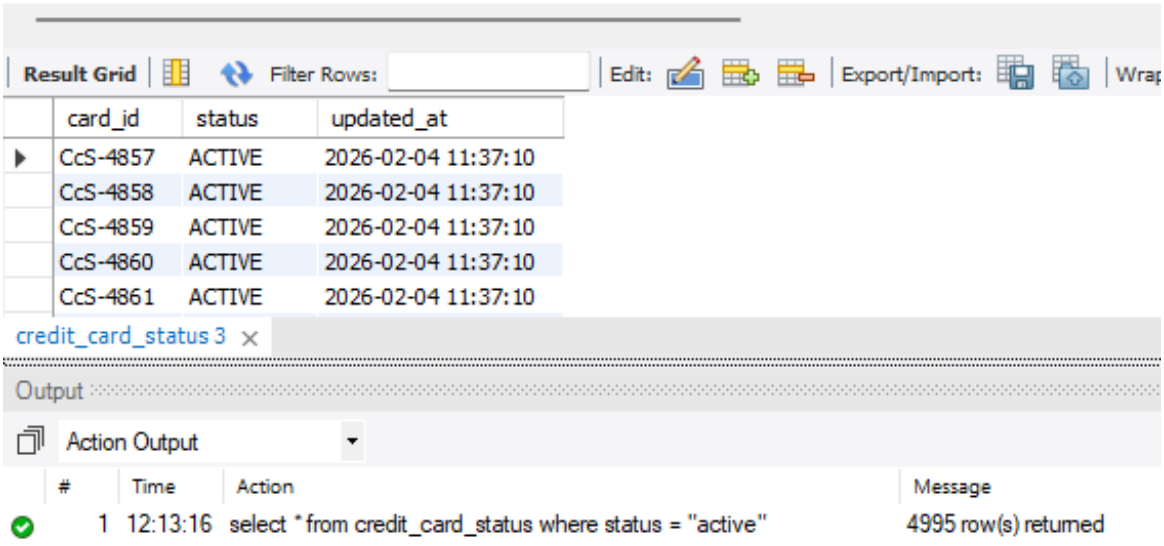
Action Output

#	Time	Action	Message
1	13:52:09	alter table credit_card_status add constraint fk_credit_card_id foreign key (card_id) references credit_cards(id)...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
2	13:53:53	alter table credit_card_status add constraint uq_credit_card_status_card unique (card_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Ejercicio 1

¿Cuántas tarjetas están activas?

```
1 •  select * from credit_card_status
2    where status = "active";
```



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

card_id	status	updated_at
CcS-4857	ACTIVE	2026-02-04 11:37:10
CcS-4858	ACTIVE	2026-02-04 11:37:10
CcS-4859	ACTIVE	2026-02-04 11:37:10
CcS-4860	ACTIVE	2026-02-04 11:37:10
CcS-4861	ACTIVE	2026-02-04 11:37:10

credit_card_status 3

Output

Action Output

#	Time	Action	Message
1	12:13:16	select * from credit_card_status where status = "active"	4995 row(s) returned

Nivel 3

Crea una tabla con la que podamos unir los datos del nuevo archivo products.csv con la base de datos creada, teniendo en cuenta que desde transaction tienes product_ids. Genera la siguiente consulta:

La creación de la tabla para unir los datos del nuevo archivo products.csv con la base de datos creada se realizó al inicio del todo

Ejercicio 1

Necesitamos conocer el número de veces que se ha vendido cada producto.

```
652 •  select p.id as idProducto, p.product_name as nombreProducto, count(t.id) as conteoVentas
653    from products p
654    join products_transactions pt
655    on p.id = pt.products_id
656    join transactions t
657    on pt.transactions_id = t.id
658    group by idProducto, nombreProducto;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

idProducto	nombreProducto	conteoVentas
1	Direwolf Stannis	2468
10	Karstark Dorne	2571
100	south duel	2517
11	Karstark Dorne	2573
12	duel Direwolf	2558
13	palpatine chewbacca	2560
14	Direwolf	2496
15	Stannis warden	2535
16	the duel warden	2608
17	skywalker ewok sith	2527

Result 13

Output

Action Output

#	Time	Action	Message
1	17:12:46	select p.id as idProducto, p.product_name as nombreProducto, count(t.id) as conteoVentas from products p j...	100 row(s) returned